STARTUP PROJECT TECHNICAL DOCUMENTATION

Enterprise CRM, HRMS & ERP Management Platform

Complete Architecture, Multi-Module Engineering Design & Operations Manual

A comprehensive production-grade enterprise operating platform unifying Customer Relationship Management (CRM), Human Resources (HRMS), Biometric Attendance Kiosk, Central Payroll Engine, Employee Self-Service (ESS), Double-Entry Accounting, Procurement, and Project Resource Delivery into a synchronized real-time web application.

PROJECT NAME	**SYSTEM RELEASE VERSION**
Enterprise CRM, HRMS & ERP Management Platform (ERP Suite)	v2.4.0-production
DOCUMENT VERSION	**CURRENT STATUS**
v1.0 (Master Handover Edition)	Production Ready / Active Maintenance
PREPARED BY	**LAST UPDATED DATE**
Core Architecture & Engineering Team	October 31, 2026

2 Project Summary

2.1 Purpose of the Project

The **Enterprise CRM, HRMS & ERP Management Platform** is an all-in-one, multi-tenant capable business operating platform. It bridges the deep operational gaps between customer acquisition, workforce administration, and financial accounting. By replacing multiple disconnected third-party tools, the platform acts as the organizational single source of truth across 25 dedicated user interfaces and 41 backend API routers.

2.2 Business Problem It Solves

Fast-growing startups and enterprises frequently run into severe operational bottlenecks caused by software fragmentation. Organizations typically maintain separate SaaS subscriptions for CRM, payroll calculation, attendance tracking, project delivery, and accounting. This causes:

- **Data Synchronization Latency:** Sales teams win deals, but operational teams must manually re-create client projects, quotes, and billing ledgers.
- **Attendance & Payroll Disconnect:** Manual export of punch-clock spreadsheets into payroll results in inaccurate Loss-of-Pay (LOP) calculations and non-compliance with statutory tax deductions.
- **Excessive Subscription Costs:** Compounding per-seat monthly subscription fees drain operational cash flow.
- **Audit & Access Exposure:** Lack of centralized role-based access control creates severe security risks and prevents transparent compliance audits.

This platform directly solves these problems through an integrated dual-PostgreSQL database architecture, automated database triggers, real-time WebSocket state distribution, and standardized RESTful services.

2.3 Intended Users

USER PERSONA	SYSTEM ROLE	PRIMARY RESPONSIBILITIES & CAPABILITIES
Executive / Super Admin	`super_admin`	Company-wide oversight, financial KPIs, module-level permission assignments, audit log inspection, and system preferences.
HR Administrator	`hr_admin`	Employee lifecycle (onboarding, branches, designations), shift scheduling, leave adjudication, and statutory payroll execution.
Finance & Accounts	`finance`	General ledger management, Chart of Accounts, tax invoice generation, banking reconciliations, and expense reimbursement approvals.
Sales Director & Team	`sales`	Commercial lead qualification, Kanban pipeline management, opportunity progression, quotations, and customer contracts.
Project Manager	`manager`	Project milestone planning, resource allocations, group workflows, task checklists, and employee timesheet reviews.
Employee (ESS)	`employee`	Dedicated `/employee` portal: Biometric clock-in/out, leave applications, task tracking, document vault, and payslip downloads.

2.4 Current Status

> **Status: Production Ready / Active Maintenance** — All 25 frontend modules are operational; 41 backend API route groups are active; 23 sequential SQL migrations have been executed idempotently; and 9 automated QA testing suites (45+ tests) validate end-to-end functionality.

3 Business Context

3.1 Why Was This Project Built?

The system was conceived and engineered to establish sovereign technological independence and operational efficiency for high-velocity modern organizations. When an organization scales past 50 employees, disconnected spreadsheets and disparate SaaS solutions fail to provide timely data. A unified ERP was built to eliminate manual human data transfers, enforce statutory legal compliances programmatically, and provide real-time executive visibility into organizational profitability.

3.2 Core Problems Solved

1. **Lead-to-Cash Workflow Automation:** Converting a closed-won sales lead automatically creates a registered Customer profile, spawns a Delivery Project with predefined milestones, and enables instant quotation generation without duplicate data entry.
2. **Biometric-to-Payslip Automation:** Punch events captured by the biometric kiosk or employee web interface stream directly into the attendance ledger. At month-end, the payroll engine programmatically evaluates worked days, unapproved absences, statutory PF/ESI caps, and LOP deductions to yield locked payslips.
3. **Immutable Enterprise Auditability:** Every financial ledger journal entry, customer status update, and credential modification is logged with actor identification, client IP, and UTC timestamps.

3.3 Expected Business Value

KEY BUSINESS METRIC	QUANTIFIABLE IMPACT	OPERATIONAL VALUE DELIVERED
Admin Processing Time	**~40% Reduction**	Automated leave balance deductions, 1-click monthly payroll runs, and rapid PDF payslip generation save over 120 administrative hours each month.
Data Synchronization Silos	**100% Eliminated**	Shared PostgreSQL schemas eliminate disparate customer files and disconnected employee records across departments.
Statutory Compliance Accuracy	**100% Deterministic**	Provident Fund (PF ₹1,800 cap), ESI (0.75%), Professional Tax (PT), and TDS withholdings computed strictly to Indian statutory tax standards.
SaaS Licensing Overhead	**Zero Per-Seat SaaS**	Replaces 5+ external recurring monthly subscription contracts with a self-hosted, scalable web application.

4 System Overview

The platform implements a decoupled, modern **Three-Tier Web Architecture** designed for high throughput, security, and fault isolation:



4.1 Presentation Tier

Built as a responsive Single Page Application (SPA) leveraging React 18, TypeScript, and Tailwind CSS. State is managed centrally via the `AppContext` provider, decoupling UI components from network requests and ensuring optimistic UI updates.

4.2 Application Tier

An Express.js server operating under native ES Modules (`"type": "module"`). It processes incoming requests through a strict middleware chain: CORS verification, rate-limiting on sensitive authentication routes, JWT

signature verification, and dynamic module-level authorization checks.

4.3 Persistence Tier

Data persistence is divided into two separate PostgreSQL instances: **HRMS** and **CRM**. This dual-pool architecture guarantees complete data isolation between internal workforce data (salaries, biometric logs, HR policies) and external client operations (deals, pipelines, invoices).

5 Technology Stack

LAYER	TECHNOLOGY	VERSION	ROLE & RATIONALE
Frontend Core	React • TypeScript	18.3 • 5.5	Component modularity with compile-time type safety across all 25 operational modules.
Build Engine	Vite	6.4.3	Lightning-fast development HMR and tree-shaken, optimized Rollup production builds.
Styling / UI	Tailwind CSS • Lucide	3.4.1 • 1.16	Utility-first responsive layout with consistent enterprise design tokens and SVG icons.
Backend Runtime	Node.js	v18+ / v20+	Asynchronous, non-blocking event-driven runtime handling concurrent REST and WebSocket traffic.
API Server	Express.js (ESM)	4.21.2	Minimalist, unopinionated web framework using standard ES Modules (`"type": "module"`).
Database	PostgreSQL	14+ / 16 Alpine	ACID relational database engine with advanced indexing, JSONB storage, and PL/pgSQL triggers.
DB Driver	node-postgres (`pg`)	8.13.1	High-performance native connection pooling executing raw parameterized SQL queries ($1, $2).
Authentication	jsonwebtoken • bcryptjs	9.0.2 • 2.4.3	Stateless signed JWT bearer tokens and adaptive 10-salt-round cryptographic password hashing.
Validation	Zod	3.24.2	Composable TypeScript-first runtime schema validation for incoming HTTP payload parameters.
Real-Time Engine	ws (WebSockets)	8.18.0	Full-duplex RFC 6455 bidirectional socket communication for instant alert and roster broadcasts.
File Storage	Multer	2.4.0	Multipart form streaming handler storing attachments with extension and MIME-type gating.
Email Service	Nodemailer	6.10.1	SMTP transport for automated dispatch of payslip PDFs, leave alerts, and system notices.
PDF Engine	pdf-lib	1.17.1	Direct binary PDF document synthesis for formal 4-page software test reports and employee payslips.
Containerization	Docker • Compose	Compose v2	Reproducible multi-container orchestration for backend, frontend, and dual database nodes.

Strategic Rationale Behind Key Technology Choices

- **Raw Parameterized SQL over ORMs:** Using parameterized node-postgres queries eliminates ORM performance overhead, prevents N+1 query bottlenecks, and allows utilizing native PostgreSQL triggers for automatic timestamping and attendance duration math.
- **Dual Database Isolation:** Isolates sensitive employee compensation and biometric data from sales CRM records, allowing independent database backup, scaling, and access auditing.

6 Major Modules

1. HRMS — Core Employee Lifecycle

HRMS DB

Purpose: Comprehensive administration of organizational structure and employee records.

Responsibilities: Onboarding, employee profile maintenance, role assignment, departmental classification, branch allocation, and exit status transitions.

Main Features: Auto-generated sequential employee codes (`EMP-001`); emergency contacts; PAN/Aadhaar compliance fields; multi-branch infrastructure; one-click employee status toggling (Active, On Probation, Notice, Terminated).

Dependencies: `employees` , `departments` , `designations` , `branches` tables.

2. Attendance Management & Biometric Kiosk

HRMS DB

Purpose: Accurate recording of daily work hours, biometric punch events, and shift compliance.

Responsibilities: Enforcing shift timings, tracking biometric inputs, computing late marks, overtime calculation, and providing a full-screen interactive Kiosk interface.

Main Features: Fullscreen kiosk mode with PIN authentication and webcam visual verification; automated calculation of daily work hours via database triggers (`clock_out - clock_in`); shift master configuration; real-time admin roster updates via WebSockets.

Dependencies: `attendance_logs` , `shifts` , `employees` , `ws` WebSocket broadcast.

3. Leave Management Engine

HRMS DB

Purpose: Digital administration of employee time-off, statutory leave entitlements, and approvals.

Responsibilities: Balance decrementing, leave history tracking, multi-level manager approval, and holiday calendar integration.

Main Features: Configurable leave quotas (Casual Leave - CL, Sick Leave - SL, Privilege Leave - PL); automated balance checks preventing negative balances; real-time approval/rejection triggers; approved unpaid leaves automatically flagged as Loss of Pay (LOP) for payroll processing.

Dependencies: `leave_requests` , `leave_balances` , `employees` , `payroll_runs` .

4. Central Payroll Engine

HRMS DB

Purpose: Autonomous, compliant payroll calculation, payslip delivery, and statutory tracking.

Responsibilities: Generating gross-to-net pay computations, enforcing statutory deductions, and locking monthly payroll runs.

Main Features: Indian statutory compliance: PF (12% of Basic capped at ₹1,800/month), ESI (0.75% for gross < ₹21,000), Professional Tax (PT), and TDS withholdings; Loss of Pay formula (`(Gross / Total Days) * LOP Days`); instant payslip PDF generation via `pdf-lib` .

Dependencies: `employee_salaries` , `payroll_runs` , `payslips` , `chart_of_accounts` .

5. Employee Self-Service (ESS) Portal

HRMS DB

Purpose: Self-service portal empowering employees to access personal data without admin dependency.

Responsibilities: Enabling clock-in/out, leave applications, attendance regularization, and document access.

Main Features: Dedicated isolated route at `/employee`; interactive punch widget with live stopwatch timer; leave balance tracker; payslip download archive; company document viewer.

Dependencies: JWT token claims with `role: "employee"`, `routes/ess.js`.

6. CRM & Lead Pipeline (Kanban)

CRM DB

Purpose: Commercial lead acquisition, qualification tracking, and sales cycle management.

Responsibilities: Lead ingestion, lead scoring, deal stage tracking, activity logging, and follow-ups.

Main Features: Drag-and-drop Kanban pipeline (New → Contacted → Qualified → Proposal → Negotiation → Won → Lost); scheduled follow-up reminders; 1-click conversion from Won Lead into active Customer record and linked onboarding Project.

Dependencies: `crmPool`, `leads`, `customers`, `opportunities`, `crm_activities`.

7. Sales Operations (Quotations, Orders & Invoices)

CRM DB

Purpose: Commercial transaction generation, tax calculation, and payment lifecycle tracking.

Responsibilities: Price quotation authoring, sales order confirmation, and tax invoice generation.

Main Features: Line-item builder with tax rate calculations (GST/VAT); quotation revision history; sales order tracking; invoice status ledger (Draft, Issued, Partially Paid, Paid, Void).

Dependencies: `quotations`, `sales_orders`, `crm_invoices`, `products`, `customers`.

8. Finance, General Ledger & Expenses

HRMS DB

Purpose: Comprehensive enterprise bookkeeping and cash flow tracking.

Responsibilities: Double-entry accounting, bank ledger reconciliation, and staff expense processing.

Main Features: Standard Chart of Accounts (COA); automatic journal entry creation linked to payroll runs and sales invoice payments; bank account reconciliation; staff expense claims with receipt image attachments.

Dependencies: `accounts`, `general_ledger`, `bank_accounts`, `expense_claims`.

9. Projects & Resource Delivery

CRM DB

Purpose: Client delivery governance, milestone scheduling, and group collaboration.

Responsibilities: Tracking project scope, repository links, delivery phases, and team assignments.

Main Features: Project details view with repository URLs, budgets, and milestone progress; Resource Allocation Panel mapping staff to projects with percentage allocation and duration; team chat and phase workflows.

Dependencies: `projects`, `project_allocations`, `project_groups`, `team_chat_messages`.

10. Timesheets & Enterprise Task Board

HRMS DB

Purpose: Daily operational task distribution and billable timesheet tracking.

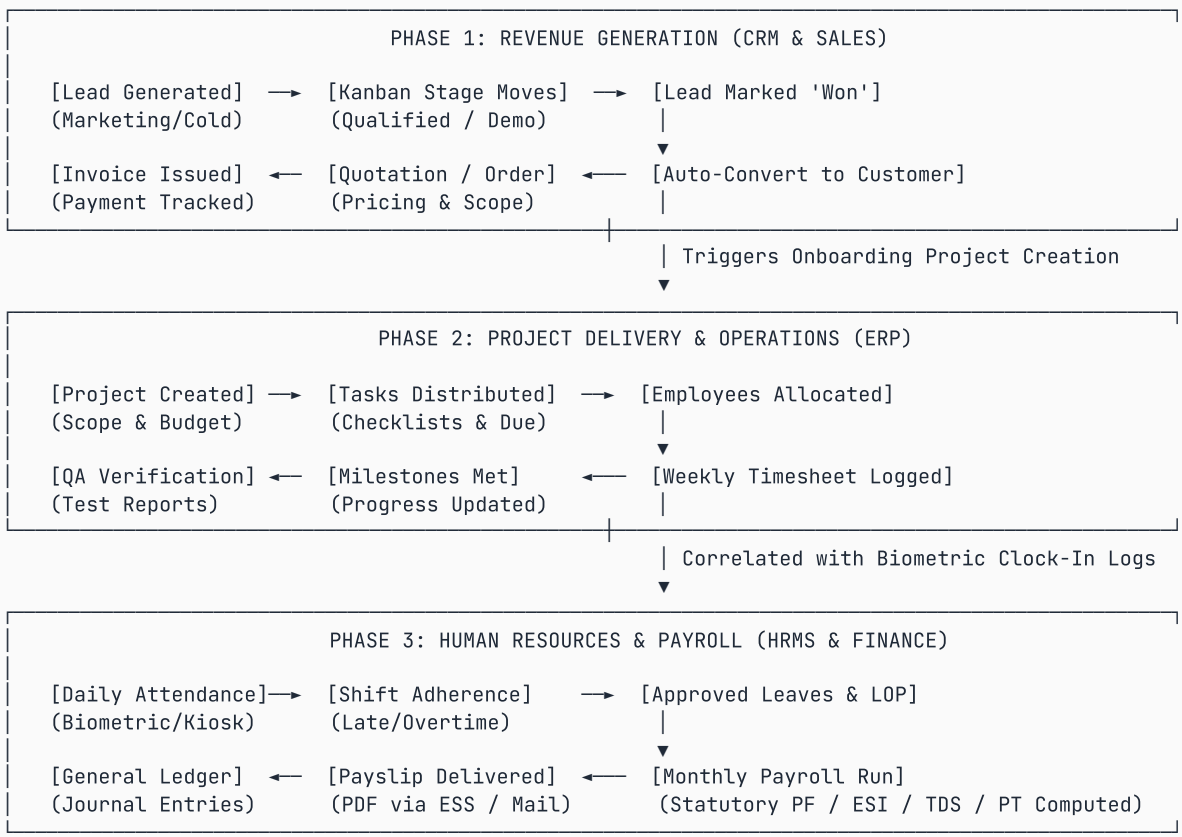
Responsibilities: Task assignments, checklist verification, time logging, and manager approvals.

Main Features: Standalone weekly timesheet submission with project, task, and billable hour tagging; administrative timesheet review and approval workflow; enterprise task board with subtask checklists and file attachments.

Dependencies: `timesheets`, `tasks`, `task_attachments`, `employees`, `projects`.

7 System Workflow

The platform integrates commercial sales, operational delivery, human resource management, and financial accounting into an end-to-end continuous loop:

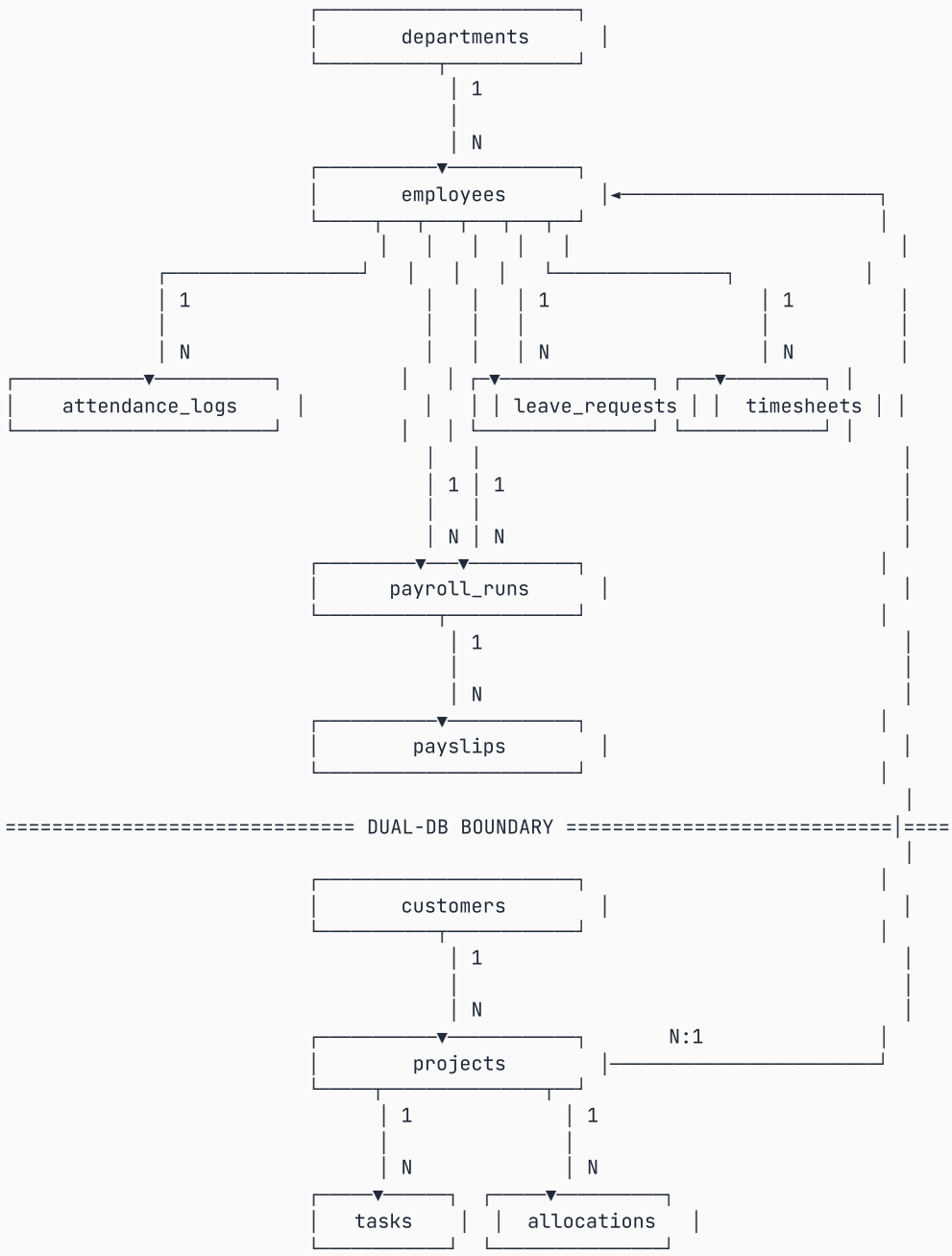


Step-by-Step Lifecycle Description:

1. **Prospecting & Closing:** Inquiries enter the CRM pipeline as Leads. Sales personnel record activities and advance deals through Kanban columns. When an opportunity is won, clicking "Convert" spawns a Customer record and creates a client Project.
2. **Resource Provisioning:** Project Managers allocate developers and engineers to the project using the Allocation Panel, defining percentage allocation and timeframe. Tasks are assigned with subtask checklists.
3. **Time & Attendance Tracking:** Staff log into the ESS portal or physical Kiosks to record daily attendance punches. Weekly billable hours are submitted through the Standalone Timesheet module for supervisor review.
4. **Payroll Computation & Ledger Posting:** At the end of the pay cycle, HR executes the payroll run. The backend evaluates actual attendance against shift rosters, deducts Loss of Pay for unapproved leaves, applies statutory deductions (PF, ESI, PT, TDS), drafts locked payslips, and posts journal debits/credits directly into the General Ledger.

8 Data Model

The data layer is segregated across two PostgreSQL databases. Below is the relational entity map showing the primary business entities and their foreign key relationships:



Key Database Entity Specifications:

ENTITY NAME	DATABASE	KEY ATTRIBUTES	DESCRIPTION & RELATIONAL LINKAGES
`employees`	HRMS	`id, employee_id, email, password_hash,`	Master personnel table. Linked to departments, designations, branches, and leave balances.

ENTITY NAME	DATABASE	KEY ATTRIBUTES	DESCRIPTION & RELATIONAL LINKAGES
		`role, department_id, status`	
`attendance_logs`	HRMS	`id, employee_id, date, clock_in, clock_out, total_hours, status`	Daily biometric logs. `total_hours` calculated automatically by database trigger.
`leave_requests`	HRMS	`id, employee_id, leave_type, start_date, end_date, status, l_days`	Time-off applications with balance checking and manager approval workflows.
`payroll_runs`	HRMS	`id, month, year, total_gross, total_net, status, processed_at`	Monthly payroll run master record tracking organization-wide disbursement state.
`payslips`	HRMS	`id, employee_id, payroll_run_id, basic, hra, pf, esi, pt, tds, net_salary`	Itemized gross-to-net pay vouchers with statutory deduction breakdowns.
`leads`	CRM	`id, name, company, email, status, value, source, assigned_to`	Prospect tracking through Kanban pipeline stages (New → Won → Lost).
`customers`	CRM	`id, customer_name, customer_code, email, phone, industry, address`	Master client registry created upon winning leads or manual onboarding.
`projects`	CRM	`id, code, name, client, customer_id, budget, status, repository_url`	Delivery project tracking scope, milestone dates, and development repositories.

9 External Integrations

SERVICE / PROTOCOL	INTEGRATION CHANNEL	OPERATIONAL PURPOSE
SMTP Mail Server	Nodemailer (TLS/STARTTLS on port 587/465)	Automated delivery of system alerts, leave approval confirmations, and employee monthly payslip PDF attachments.
OpenAI API	Official `openai` SDK (REST API)	Intelligent sales proposal refinement, candidate resume evaluation, and automated task summary generation.
Biometric Punch Kiosk	HTTP POST Webhook / REST Endpoint	Biometric fingerprint and facial recognition scanners submit punch events via `/api/attendance/kiosk-punch` with IP verification.
PDF Vector Synthesis	In-Memory `pdf-lib` Library	High-speed, zero-dependency server-side and client-side compilation of formal 4-page Software Test Reports and Payslips.
WebSocket Protocol	Native RFC 6455 `ws` Library	Maintains persistent full-duplex socket connections, pushing instant live updates to administrative dashboards and employee self-service screens.

10 Configuration

The application is configured via environment variables. Below is the sanitized configuration specification:

```
# ── Server Configuration ──────────────────────────────
PORT=5000
NODE_ENV=development                        # 'development' | 'production' | 'test'

# ── HRMS Database (Employees, Payroll, Attendance) ────
DB_USER=postgres
DB_PASSWORD=YOUR_POSTGRESQL_PASSWORD
DB_HOST=localhost                           # e.g., 'localhost' or 'hrms-db' in Docker
DB_NAME=HRMS
DB_PORT=5432

# ── CRM Database (Leads, Customers, Sales, Projects) ──
CRM_DB_USER=postgres
CRM_DB_PASSWORD=YOUR_POSTGRESQL_PASSWORD
CRM_DB_HOST=localhost                       # e.g., 'localhost' or 'crm-db' in Docker
CRM_DB_NAME=crm
CRM_DB_PORT=5432

# ── Authentication Security ───────────────────────────
JWT_SECRET=YOUR_SECURE_RANDOM_LONG_SECRET_KEY_2026

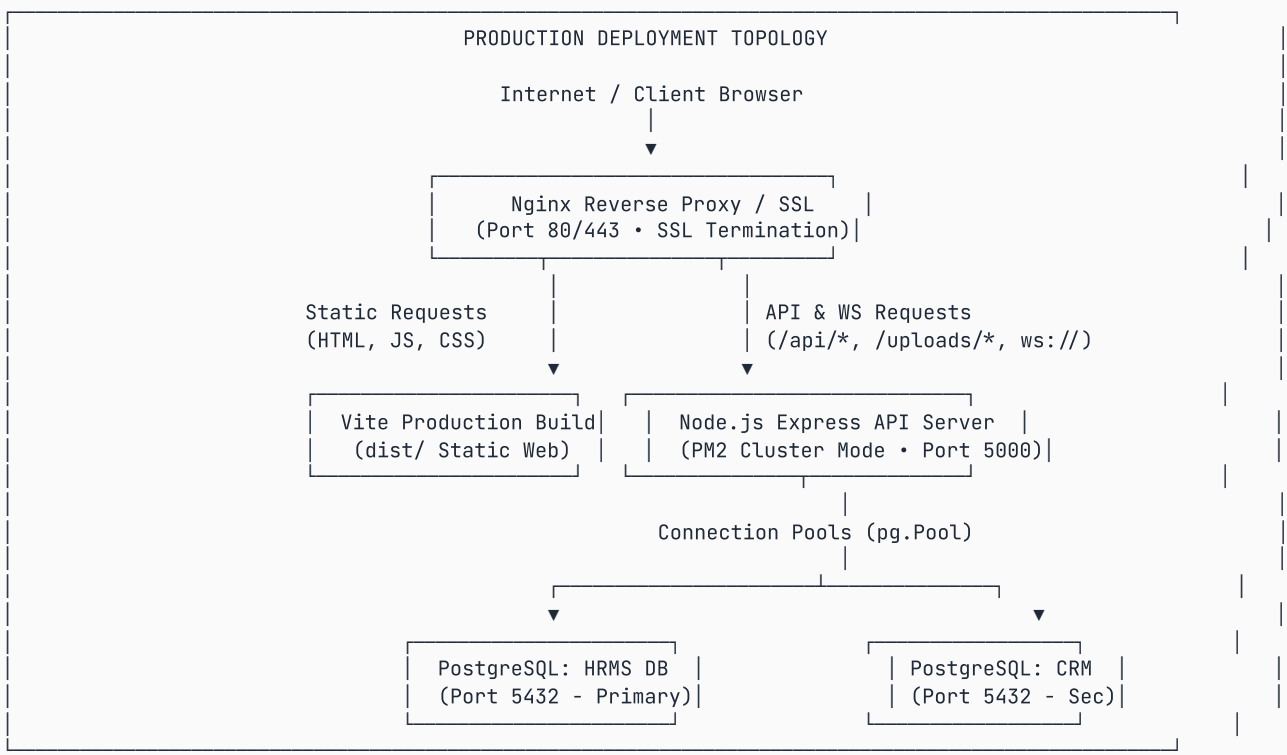
# ── SMTP Email Service ────────────────────────────────
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587                               # 587 (STARTTLS) or 465 (SSL)
SMTP_SECURE=false                           # true for port 465, false for 587
SMTP_USER=your_organization_email@gmail.com
SMTP_PASS=your_app_specific_password
MAIL_FROM="Enterprise ERP" <your_organization_email@gmail.com>

# ── AI Integrations (Optional) ────────────────────────
OPENAI_API_KEY=sk-proj-YOUR_OPENAI_API_KEY_HERE
```

Key Configuration Files:

- `backend/.env`: Primary server configuration and database credentials.
- `backend/db/pool.js`: Initializes the two independent `pg.Pool` connection instances.
- `frontend/vite.config.ts`: Configures Vite bundling and proxy rules forwarding `/api` requests to port 5000.
- `docker-compose.yml`: Production container definitions for frontend, backend, and dual PostgreSQL instances.

11 Deployment Overview



11.1 Development Environment

- **Backend:** Executed natively with Node.js watch mode (`npm run dev`) on port 5000.
- **Frontend:** Hosted on Vite development server with Hot Module Replacement on port 5173.
- **Databases:** Hosted on local PostgreSQL service running on port 5432.

11.2 Staging (Dockerized) Environment

- Orchestrated via `docker-compose.yml` exposing isolated container instances:
  - `crm-frontend` — Port 3001
  - `crm-backend` — Port 5001
  - `hrms-db` — PostgreSQL container on port 5434
  - `crm-db` — PostgreSQL container on port 5435

11.3 Production Deployment Topology

- **Reverse Proxy:** Nginx handles SSL termination, HTTP/2 multiplexing, static asset caching, and rate limiting.
- **Process Manager:** PM2 executes the Express backend in cluster mode (`pm2 start index.js -i max`), providing zero-downtime rolling reloads and automatic restarts on uncaught exceptions.
- **Database Cluster:** Dedicated PostgreSQL 16 server with automated Write-Ahead Logging (WAL) archiving and scheduled nightly logical backups.

12 Project Structure

```
CRM/
├── backend/                          # Express.js Backend API Service
│   ├── config/                       # System configuration & loaders
│   ├── controllers/                  # Domain controller routines
│   ├── db/                           # Database connectivity & migrations
│   │   ├── migrations/               # 23 sequential SQL migration files
│   │   ├── migrator.js               # Idempotent schema migration runner
│   │   └── pool.js                   # Dual pg.Pool instances (hrmsPool, crmPool)
│   ├── middleware/                   # Request middleware chain
│   │   ├── auth.js                   # JWT token authentication
│   │   ├── moduleAuth.js             # Role & module authorization guards
│   │   └── errorHandler.js           # Centralized error formatter
│   ├── routes/                       # 41 REST router definitions
│   ├── services/                     # Business services (payroll, email, pdf)
│   ├── tests/                        # Master runner & 9 test suites
│   ├── uploads/                      # File storage (tasks, docs, timesheets)
│   ├── utils/                        # Shared utilities & WebSocket server
│   ├── validators/                   # Zod payload validation schemas
│   ├── index.js                      # Application entry point & boot sequence
│   └── package.json                  # Backend dependencies
│
├── frontend/                         # React 18 TypeScript Single Page App
│   ├── public/                       # Static assets, logos, and favicons
│   ├── src/
│   │   ├── app/                      # Navigation routes and layout wrappers
│   │   ├── components/               # Shared UI primitives (modals, buttons, tables)
│   │   ├── context/                  # Global state via AppContext.tsx
│   │   ├── modules/                  # 25 domain-driven feature modules
│   │   │   ├── attendance/           # Kiosk, attendance rosters, regularization
│   │   │   ├── crm/                  # Lead Kanban, customers, follow-ups
│   │   │   ├── documents/            # DMS categories, file list, preview modals
│   │   │   ├── employee/             # Dedicated ESS self-service portal
│   │   │   ├── hrms/                 # Employee directory, departments, branches
│   │   │   ├── payroll/              # Salary runs, CTC config, payslip generator
│   │   │   ├── projects/             # Delivery views, project allocations, groups
│   │   │   ├── tasks/                # Enterprise task boards & subtasks
│   │   │   ├── testing/              # QA test suites & test reports
│   │   │   └── timesheets/           # Standalone weekly timesheet entries
│   │   ├── services/                 # Typed API client services (apiService.ts)
│   │   ├── types/                    # Global TypeScript interfaces
│   │   ├── App.tsx                   # Root React router switch
│   │   └── main.tsx                  # DOM entry point
│   ├── vite.config.ts                # Vite build configuration & reverse proxy
│   └── package.json                  # Frontend dependencies
│
└── docs/                             # Complete documentation repository
    ├── 1_Project_Documentation.md        # High-level architecture overview
    ├── 2_Source_Code_Handover_Report.md  # Detailed routing & codebase inventory
    ├── 3_Project_Setup_Guide.md          # Step-by-step developer runbook
    ├── 4_Technical_Documentation.md      # Security, database triggers, payroll logic
    ├── 5_Database_and_API_Documentation.md # Full table schemas and endpoint inventory
    ├── 6_Testing_Report.md               # QA strategy & test run commands
    ├── 7_Project_Status_and_Pending_Work.md # Known issues & feature backlog
    ├── 8_Final_Project_Handover_Summary.md  # Master checklist & handover summary
    ├── 9_Testing_Reports_All_Types.md    # 9-suite test report documentation
    └── Startup_Project_Documentation.pdf # ← Official Compiled Technical PDF Manual
```

13 Known Limitations

13.1 Scalability Limits

In-Memory WebSocket Hub: The current real-time notification engine utilizes Node's built-in `ws` library within process memory. This efficiently handles up to 5,000 concurrent sockets on a standard single-server deployment. Beyond this threshold, an external message broker (such as Redis Pub/Sub) must be implemented to broadcast socket events across multi-instance worker processes.

Local File System Storage: File attachments (documents, task files, contracts) are written to `backend/uploads/` on the host machine. Multi-server cloud deployments require transitioning to an object storage provider (e.g., AWS S3 or Cloudflare R2).

13.2 Technical Debt

- **Dual Module Format in Test Harness:** While the application backend is authored in native ES Modules (`"type": "module"`), several test scripts in `tests/` use CommonJS conventions.
- **Database Trigger Dependencies:** Core attendance calculations and updated-at timestamps are governed by PL/pgSQL database triggers. While computationally fast, developers must inspect SQL migrations rather than application code to debug trigger-based updates.

13.3 Postponed Features

- **Native Mobile Applications:** Currently delivered via a mobile-optimized responsive web interface (ESS). Dedicated native iOS/Android applications using React Native are planned for future versions.
- **In-App Video Conferencing:** The recruitment module stores external meeting URLs (Google Meet/Zoom) rather than providing in-browser WebRTC video calls.

14 Maintenance Notes

14.1 Scheduled Operations

- **Daily Attendance Reconciliation (00:01 AM Cron):** Evaluates punch logs against shift schedules and marks employees without clock-ins as "Absent".
- **Monthly Payroll Pre-Draft (25th of Month):** Generates preliminary payroll calculations, incorporating attendance Loss-of-Pay deductions for HR review.
- **Weekly Database Maintenance:** Executes `VACUUM ANALYZE` on PostgreSQL every Sunday at 02:00 AM to optimize query plans and reclaim storage.

14.2 Backup Strategy

BACKUP TYPE	FREQUENCY	RETENTION	STORAGE DESTINATION
Daily Full Dump	Every 24 hours (01:00 AM)	7 Days	Local encrypted backup volume
Weekly Archive	Every Sunday	4 Weeks	Encrypted off-site cloud storage
Monthly Snapshot	1st of every month	12 Months	Cold archival storage

14.3 Common Troubleshooting Playbook

ISSUE ENCOUNTERED	LIKELY CAUSE	IMMEDIATE REMEDIATION STEPS
`ECONNREFUSED 127.0.0.1:5432`	PostgreSQL service inactive or credentials incorrect	Start PostgreSQL service (`systemctl start postgresql`). Verify DB credentials in `backend/.env` .
`ERR_MODULE_NOT_FOUND`	Missing dependency in `node_modules`	Run `npm install` in the respective `backend/` or `frontend/` directory.
`403 Forbidden: Module Not Permitted`	User lacks permission for requested module	Assign the module to the user via Super Admin Settings or check the `employee_module_assignments` table.
WebSocket Connection Disconnected	Reverse proxy dropping upgrade headers	Ensure Nginx configuration includes `proxy_set_header Upgrade $http_upgrade;` and `proxy_set_header Connection "upgrade";` .

15 Changelog & Version History

VERSION	RELEASE DATE	KEY MILESTONES & CHANGES
`v1.0.0`	May 15, 2026	Initial prototype release. Basic employee records, rudimentary attendance logging, and simple lead table.
`v1.5.0`	July 10, 2026	Migrated to Dual-Database architecture (`HRMS` + `crm`). Introduced idempotent migration runner and JWT authentication.
`v2.0.0`	August 28, 2026	Central Payroll Engine with Indian statutory rules (PF/ESI/PT/TDS). Launched dedicated Employee Self-Service (`/employee`) portal and biometric Kiosk UI.
`v2.2.0`	September 20, 2026	Enterprise Task Board with subtask checklists. Integrated Document Management System (DMS) with Multer upload pipeline and SHA-256 checks.
`v2.3.0`	October 14, 2026	Integrated QA Testing Management module. Added Project Allocations panel and Standalone Timesheets system with manager approval gates.
`v2.4.0-prod`	October 30, 2026	**Current Production Release.** Master 9-suite testing harness (45+ tests), Nodemailer SMTP transactional integration, Proposal Refinement AI workflows, and security hardening.

16 References & Separate Documents

16.1 Internal Documentation Library

DOCUMENT NAME	CONTENTS & SCOPE
`1_Project_Documentation.md`	High-level platform overview, business problems, user personas, and module inventories.
`2_Source_Code_Handover_Report.md`	Granular source code inventory, 41 API router paths, and middleware sequences.
`3_Project_Setup_Guide.md`	Step-by-step developer onboarding, PostgreSQL setup, and Docker deployment guide.
`4_Technical_Documentation.md`	Architectural deep-dive covering JWT security, database triggers, and payroll formulas.
`5_Database_and_API_Documentation.md`	Full entity relationship models, 23 migration histories, and complete API endpoint catalog.
`6_Testing_Report.md`	QA strategy, test suite categorization, and automated testing run instructions.
`7_Project_Status_and_Pending_Work.md`	Module completion audit, known issues matrix, and prioritized backlog.

DOCUMENT NAME	CONTENTS & SCOPE
`8_Final_Project_Handover_Summary.md`	Master handover checklist, deployment credentials, and operational runbook.
`9_Testing_Reports_All_Types.md`	In-depth test execution report covering all 9 software testing disciplines.

16.2 Notice on Separate Operational Documents

Notice: In accordance with enterprise software engineering standards, the following companion manuals should be maintained and referenced as separate standalone documents for distinct operational teams:

- **API Documentation:** Detailed Swagger/OpenAPI 3.0 specification covering every request/response JSON payload and HTTP status code.
- **Database Schema Guide:** Full data dictionary listing every table, column data type, default value, index, and trigger definition.
- **Deployment & DevOps Guide:** Production infrastructure runbook with Nginx configs, SSL renewal scripts, and Docker container maintenance.
- **Testing & QA Guide:** Test case catalog, test data seeding scripts, and regression testing protocols.
- **Developer Onboarding Guide:** Day-1 developer guide covering local environment setup, Git branching models, and pull request conventions.
- **User Manual & Training Guides:** Role-specific operating guides for HR Admins, Accountants, Sales Staff, and General Employees.